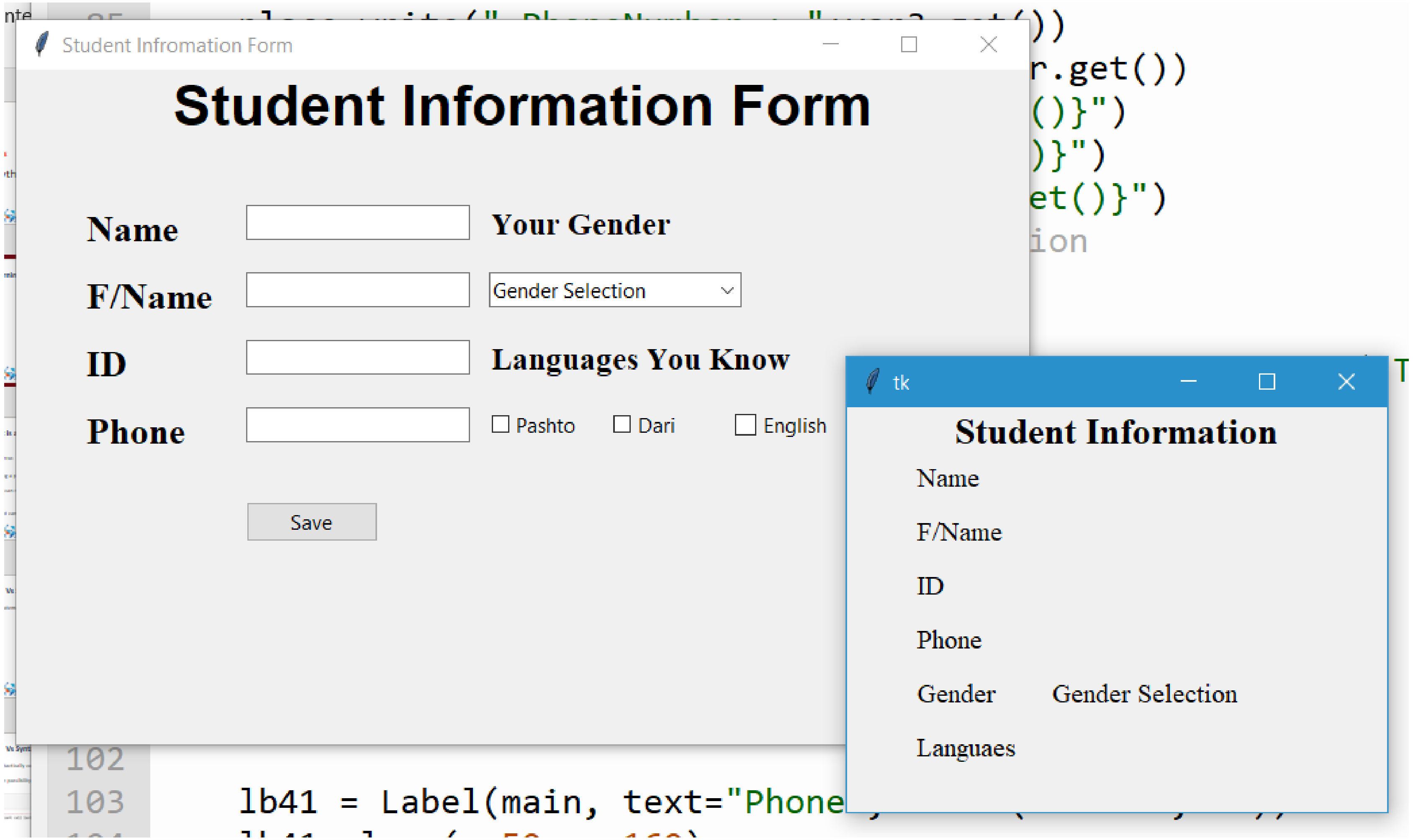
Homework


Student Infromation Form
Student Information Form
Name
F/Name
ID
Phone
Your Gender
Gender Selection
Languages You Know
Pashto
Dari
English
Save
tk
Student Information
Name
F/Name
ID
Phone
Gender
Gender Selection
Languaes
102
103
lb41 = Label(main, text="Phone

Learning Outcomes

- Should know about What is Exception
- Learn Exception vs Syntax error
- Could Handling Exceptions
- Use of Finally block
- Could Raising Exceptions

What is an Exception

- A Python program terminates as soon as it encounters an error.
- In Python, an error can be a syntax error or an exception.
- An exception is an indication of a problem that occurs during a program’s execution.
- Exception handling enables you to create applications that can resolve (or handle) exceptions.

- It will help you write *robust* and *fault-tolerant* programs that can deal with problems and continue executing or *terminate gracefully*.

Exception Vs Syntax error

- Syntax errors occur when the parser detects an incorrect statement.

```
a=10
b=15
print (a+b))
```

```
  File "<ipython-input-2-90f1816b0bf0>", line 3
    print (a+b))
               ^
SyntaxError: invalid syntax
```

Exception Vs Syntax error

- **Exception error** is type of error which occurs whenever syntactically correct Python code results in an error.
- Python comes with various built-in exceptions as well as the possibility to create self-defined exceptions.

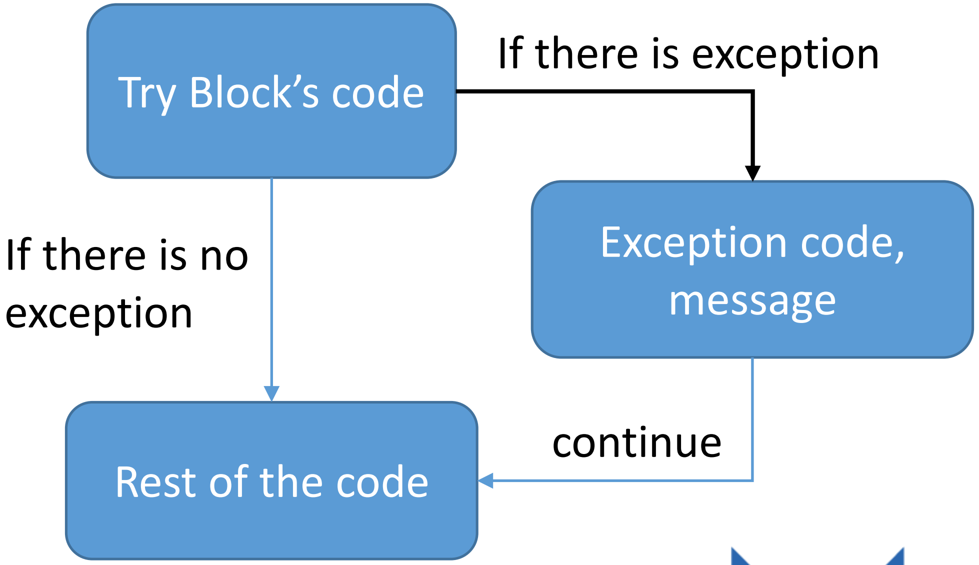
```
a=0
b=0
print (a/b)
```

```
---------------------------------------------------------------------------
ZeroDivisionError                         Traceback (most recent call last)
<ipython-input-3-2f40952ad266> in <module>
      1 a=0
      2 b=0
----> 3 print (a/b)

ZeroDivisionError: division by zero
```

Handling Exceptions

- The try and except block in Python is used to catch and handle exceptions.
- Python executes code following the try statement as a “normal” part of the program.
- This exception error will crash the program if it is unhandled.



Catching an exception

- We can use try-except block to catch any exception:
- Consider the following “index out of range exception”

```
name='Ahmad'

print(name[8])
```

```
---------------------------------------------------------------------------
IndexError                                Traceback (most recent call last)
<ipython-input-5-d78c75fce6a6> in <module>
      1 name='Ahmad'
      2 
----> 3 print(name[8])

IndexError: string index out of range
```

Catching the exception

- You have to put the code that you think it may cause a problem into a try block.

try:

Your code here

except (Type of error):

your message

Example:

```
name='Ahmad'
try:
    print(name[8])

except IndexError:
    print("Got index exception")
```

```
Got index exception
```

Catch exception without knowing the type

- Sometimes you do not know the type of exception,
- You can catch any types of exception without the type:
- try:
  - Your code
- except:
  - Your message
- Want to know the type of exception!
- try:
  - Your code
- except Exception as ex:
  - Your message plus ex

```python
name='Ahmad'
try:
    print(name[8])
except :
    print("Got exception")
```

```
Got exception
```

```python
name='Ahmad'
try:
    print(name[8])
except Exception as ex:
    print("Got exception:",ex)
```

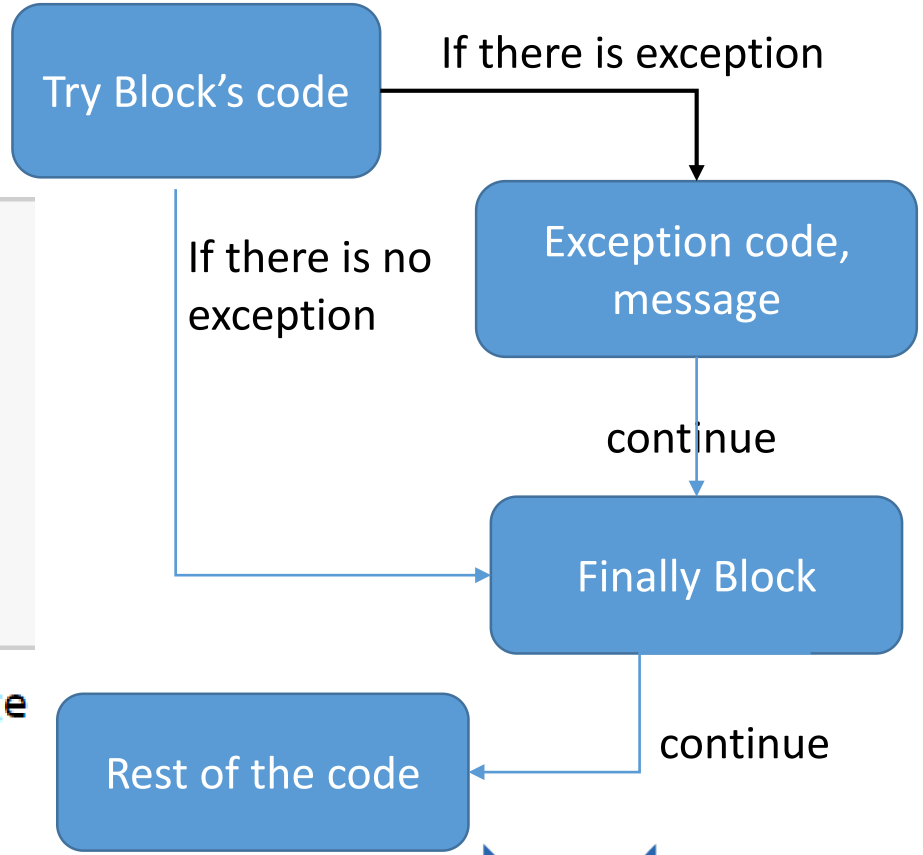
```
Got exception: string index out of range
```

Finally block

- The finally block will get executed if the exception happens or not!
- Clean up, file close, server connection

```
name='Ahmad'
try:
    print(name[8])
except Exception as ex:
    print('Exception happend: ',ex)
finally:
    print("End of the try")
```

```
Exception happend:  string index out of range
End of the try
```



How to Raise Exceptions

- We can use customized exceptions for our programs.
- Using : raise Exception (‘ Exception details here’) function. See the example

```
month=14

if month>12:
    raise Exception("Month cannot exceed 12")
```

```
---------------------------------------------------------------------------
Exception                                 Traceback (most recent call last)
<ipython-input-4-3dd87272d546> in <module>
      2 
      3 if month>12:
----> 4     raise Exception("Month cannot exceed 12")

Exception: Month cannot exceed 12
```

Common types of Exceptions

- IndexError
- TypeError
- ZeroDivisionError
- NameError
- FileNotFoundError
- ValueError
- ImportError

User Defined Exceptions

```python
class myException(Exception):
    pass
```

```python
def testException():
    raise myException

try:
    testException()
except myException:
    print("Got an user defined Exception")
```

```
Got an user defined Exception
```

```python
class myException(Exception):
    def __str__(self):
        return "This from MyException"
```

```python
raise myException
```

```
---------------------------------------------------------------------------
myException                               T
<ipython-input-39-fdf379f78c52> in <module>
----> 1 raise myException

myException: This from MyException
```

Thank You!